

Тестирование в Nuxt.js

Введение

Тестирование является важной частью разработки надежных и поддерживаемых приложений. Nuxt.js предоставляет различные инструменты и методы для тестирования компонентов, страниц, хранилища и API. В этом разделе мы рассмотрим основные подходы к тестированию Nuxt.js приложений.

Типы тестирования

Модульное тестирование (Unit Testing)

Модульные тесты проверяют отдельные функции и компоненты в изоляции.

Интеграционное тестирование (Integration Testing)

Интеграционные тесты проверяют взаимодействие между различными частями приложения.

End-to-End тестирование (E2E Testing)

E2E тесты проверяют приложение целиком, имитируя действия пользователя.

Настройка тестирования в Nuxt.js

Jest для модульного и интеграционного тестирования

```
npm install --save-dev jest @vue/test-utils vue-jest babel-jest
```

Настройка Jest в `jest.config.js`:

```
module.exports = {
  moduleNameMapper: {
    '^@/(.*)$': '<rootDir>/$',
    '^~/(.*)$': '<rootDir>/$',
  },
  moduleFileExtensions: ['js', 'vue', 'json'],
  transform: {
    '^.+\\.js$': 'babel-jest',
    '.*\\.vue$': 'vue-jest'
  },
  collectCoverage: true,
  collectCoverageFrom: [
    '<rootDir>/components/**/*.vue',
    '<rootDir>/pages/**/*.vue'
  ]
}
```

```
]
}
```

Сypress для E2E тестирования

```
npm install --save-dev cypress
```

Настройка Cypress в `cypress.json`:

```
{
  "baseUrl": "http://localhost:3000",
  "fixturesFolder": "cypress/fixtures",
  "integrationFolder": "cypress/integration",
  "pluginsFile": "cypress/plugins/index.js",
  "supportFile": "cypress/support/index.js"
}
```

Примеры тестов

Тестирование компонентов

```
// tests/components/MyComponent.spec.js
import { mount } from '@vue/test-utils'
import MyComponent from '@components/MyComponent.vue'

describe('MyComponent', () => {
  test('отображает правильный текст', () => {
    const wrapper = mount(MyComponent)
    expect(wrapper.text()).toContain('Ожидаемый текст')
  })

  test('реагирует на клик', async () => {
    const wrapper = mount(MyComponent)
    await wrapper.find('button').trigger('click')
    expect(wrapper.emitted().click).toBeTruthy()
  })
})
```

Тестирование страниц

```
// tests/pages/index.spec.js
import { mount, createLocalVue } from '@vue/test-utils'
import Vuex from 'vuex'
```

```

import Index from '@pages/index.vue'

const localVue = createLocalVue()
localVue.use(Vuex)

describe('Index page', () => {
  let store
  let wrapper

  beforeEach(() => {
    store = new Vuex.Store({
      state: {
        counter: 0
      },
      mutations: {
        increment(state) {
          state.counter++
        }
      }
    })

    wrapper = mount(Index, {
      localVue,
      store
    })
  })

  test('отображает счетчик', () => {
    expect(wrapper.text()).toContain('Счетчик: 0')
  })

  test('увеличивает счетчик при клике', async () => {
    await wrapper.find('button').trigger('click')
    expect(wrapper.text()).toContain('Счетчик: 1')
  })
})

```

E2E тестирование с Cypress

```

// cypress/integration/home.spec.js
describe('Главная страница', () => {
  beforeEach(() => {
    cy.visit('/')
  })

  it('отображает заголовок', () => {
    cy.get('h1').should('contain', 'Добро пожаловать')
  })

  it('переходит на страницу о нас', () => {

```

```
cy.get('a[href="/about"]').click()
cy.url().should('include', '/about')
cy.get('h1').should('contain', 'O нас')
})
})
```

Тестирование API

```
// tests/api/users.spec.js
const request = require('supertest')
const { setupTest } = require('@nuxt/test-utils')

describe('API /api/users', () => {
  let request

  beforeEach(async () => {
    const { server } = await setupTest({
      server: true
    })
    request = supertest(server)
  })

  test('GET /api/users возвращает список пользователей', async () => {
    const response = await request.get('/api/users')
    expect(response.status).toBe(200)
    expect(Array.isArray(response.body)).toBe(true)
  })

  test('POST /api/users создает нового пользователя', async () => {
    const userData = { name: 'Иван', email: 'ivan@example.com' }
    const response = await request.post('/api/users').send(userData)
    expect(response.status).toBe(201)
    expect(response.body).toHaveProperty('id')
    expect(response.body.name).toBe(userData.name)
  })
})
```

Тестирование в Nuxt 3

В Nuxt 3 появились новые возможности для тестирования:

```
npm install --save-dev @nuxt/test-utils vitest
```

```
// tests/pages/index.spec.js
import { describe, it, expect } from 'vitest'
```

```
import { mountSuspended } from '@nuxt/test-utils/runtime'
import Index from '~/pages/index.vue'

describe('Index page', () => {
  it('отображает приветствие', async () => {
    const wrapper = await mountSuspended(Index)
    expect(wrapper.text()).toContain('Добро пожаловать')
  })
})
```

Советы по тестированию

1. **Начинайте с модульных тестов:** Они быстрее и проще в написании
2. **Используйте моки для внешних зависимостей:** Изолируйте тесты от внешних API
3. **Тестируйте пограничные случаи:** Проверяйте обработку ошибок и крайние значения
4. **Автоматизируйте тестирование:** Интегрируйте тесты в CI/CD пайплайн
5. **Следите за покрытием кода:** Стремитесь к высокому проценту покрытия

Заключение

Тестирование является неотъемлемой частью разработки качественных Nuxt.js приложений. Правильно настроенные тесты помогают выявлять ошибки на ранних стадиях, обеспечивают уверенность при рефакторинге и документируют ожидаемое поведение приложения. Используя комбинацию модульных, интеграционных и E2E тестов, вы можете создавать надежные и поддерживаемые приложения на Nuxt.js.